

ISGA DE FES
Filière : 5ème année IDWM

Rapport de Projet Virtualisation et DevOps

Déploiement d'une Application Web avec Docker

Réalisé par :	Bilal El Haouat
Filière :	5ème année IDWM
Module :	Virtualisation et DevOps
Année universitaire :	2025 / 2026

1. Contexte et Objectifs

Ce projet s'inscrit dans le cadre du module « Virtualisation et DevOps » de la filière 5ème année IDWM. Il vise à mettre en pratique les concepts fondamentaux de la conteneurisation et des pratiques DevOps à travers la conception, la mise en place et la documentation d'une application web conteneurisée.

L'application développée, intitulée « **Mini DevOps – Gestion de produits** », est une application de gestion de stock permettant d'ajouter et de consulter des produits classés par catégorie, avec un statut de disponibilité (« En stock » / « Rupture »).

2. Architecture du Projet

L'application repose sur une architecture conteneurisée à deux services orchestrés par Docker Compose :

- **Service web** : conteneur PHP 8.2 avec Apache, exécutant l'application et exposé sur le port 8080 de l'hôte.
- **Service db** : conteneur MySQL 8.0 stockant les données (catégories et produits), exposé sur le port 3307 pour un accès externe éventuel.

Le service web dépend du service db (*depends_on*) afin de garantir un ordre de démarrage cohérent. Les données de la base sont persistées grâce à un volume Docker nommé **mysql_data**, ce qui évite leur perte lors du redémarrage des conteneurs. Le code de l'application est monté en volume (*bind mount*) dans le conteneur web, ce qui permet une modification du code sans reconstruction de l'image.

Composant	Rôle	Port exposé
web (PHP/Apache)	Interface applicative + logique métier	8080 → 80
db (MySQL)	Stockage des catégories et produits	3307 → 3306

Schéma d'architecture (texte)

Navigateur (localhost:8080) → Conteneur web (Apache + PHP, PDO) → Conteneur db (MySQL, base « devops_exam »). Le volume « mysql_data » assure la persistance des données du conteneur db.

3. Justification des Choix Techniques

Docker et Docker Compose ont été retenus car ils permettent de définir, dans un seul fichier déclaratif, l'ensemble des services nécessaires à l'application (serveur applicatif et base de données), tout en garantissant la portabilité et la reproductibilité de l'environnement sur n'importe quelle machine disposant de Docker.

PHP/Apache et MySQL ont été choisis pour leur simplicité de mise en œuvre et leur bonne intégration native (extension PDO installée directement dans le Dockerfile), ce qui convient parfaitement à une application de gestion de produits avec des opérations CRUD basiques.

Le script **deploy.sh** automatise les commandes répétitives (arrêt, reconstruction, démarrage et vérification de l'état des conteneurs), réduisant les erreurs manuelles lors du déploiement.

Le fichier **init.sql** permet une initialisation automatique du schéma et des données de démonstration dès le premier démarrage du conteneur MySQL, via le mécanisme *docker-entrypoint-initdb.d*.

4. Description des Composants du Dépôt

Fichier / Dossier	Description
Dockerfile	Image PHP 8.2 + Apache avec les extensions PDO/PDO_MySQL
docker-compose.yml	Orchestration des services web et db, volumes et réseau
app/index.php	Page principale : formulaire d'ajout et liste des produits
app/db.php	Connexion PDO à la base de données MySQL
app/style.css	Mise en forme de l'interface
db/init.sql	Création des tables catégorie/produit et données de test
scripts/deploy.sh	Script d'automatisation du déploiement local
README.md	Documentation de lancement et d'accès à l'application

5. Procédure de Déploiement

Déploiement manuel :

```
sudo docker-compose up -d --build
```

Déploiement automatisé via le script fourni :

```
./scripts/deploy.sh
```

Ce script arrête les conteneurs existants, reconstruit les images, redémarre les services et affiche leur état. Une fois le déploiement terminé, l'application est accessible à l'adresse : **http://localhost:8080**.

6. Difficultés Rencontrées

- Synchronisation du démarrage entre le conteneur web et le conteneur MySQL : le service web pouvait tenter de se connecter avant que la base de données ne soit totalement initialisée.
- Gestion de la persistance des données à travers le volume Docker afin d'éviter la perte des informations lors de la reconstruction des images.
- Configuration des ports pour éviter les conflits avec d'éventuels services MySQL déjà installés localement (port 3307 utilisé au lieu de 3306).

- Mise en place de l'extension PDO_MySQL dans l'image PHP, nécessaire pour établir la connexion entre l'application et la base de données.

7. Limites Actuelles et Perspectives d'Amélioration

Dans son état actuel, le projet couvre la partie conteneurisation de base de l'application. Afin de répondre pleinement à un cahier des charges DevOps complet, les axes d'amélioration suivants sont identifiés :

- Mettre en place un **reverse proxy Nginx** devant le service web et gérer les variables d'environnement via un fichier **.env**.
- Ajouter des **health checks** Docker Compose pour chaque service.
- Convertir le Dockerfile en version **multi-stage** afin de réduire la taille de l'image finale.
- Mettre en place une **pipeline CI/CD** (GitLab CI/CD ou GitHub Actions) avec les étapes build, test, lint et deploy, ainsi qu'une stratégie de tagging sémantique des images.
- Rédiger un script **Ansible ou Terraform** pour automatiser le provisioning du serveur de déploiement.

Ces éléments constituent la suite logique du projet pour couvrir l'ensemble des exigences du module (intégration continue, infrastructure as code).

7.1 Tableau de Correspondance avec le Cahier des Charges

Exigence	État
Dockerfile	Réalisé (mono-stage)
docker-compose.yml (app + BD)	Réalisé
Reverse proxy Nginx	Non réalisé
Fichier .env	Non réalisé
Health checks	Non réalisé
Pipeline CI/CD (build/test/lint/deploy)	Non réalisé
Tagging sémantique des images	Non réalisé
Infrastructure as Code (Ansible/Terraform)	Non réalisé
README.md	Réalisé
Rapport PDF	Réalisé (ce document)

Ce tableau présente honnêtement l'état d'avancement du projet par rapport aux quatre parties du cahier des charges. La partie « Conteneurisation » est partiellement couverte, tandis que les parties « Pipeline CI/CD » et « Infrastructure as Code » restent à développer pour une version complète du projet.

7.2 Modèle de Données

La base de données « devops_exam » repose sur un modèle relationnel simple composé de deux tables liées par une clé étrangère :

Table	Colonnes principales	Relation
categorie	id_categorie, nom_categorie	1 catégorie → N produits
produit	id_produit, nom_produit, prix, quantite, id_categorie	clé étrangère vers categorie

Cette structure permet de garantir l'intégrité référentielle : un produit ne peut exister sans être rattaché à une catégorie valide, grâce à la contrainte *fk_produit_categorie* définie dans le script **init.sql**.

8. Conclusion

Ce projet a permis de mettre en pratique les bases de la conteneurisation d'une application web avec Docker et Docker Compose, en assurant la communication entre un service applicatif PHP et une base de données MySQL, ainsi que la persistance des données et l'automatisation partielle du déploiement. Il constitue une fondation solide sur laquelle peuvent être ajoutées les briques d'intégration continue et d'infrastructure as code pour aboutir à une chaîne DevOps complète.